Proyecto de Software

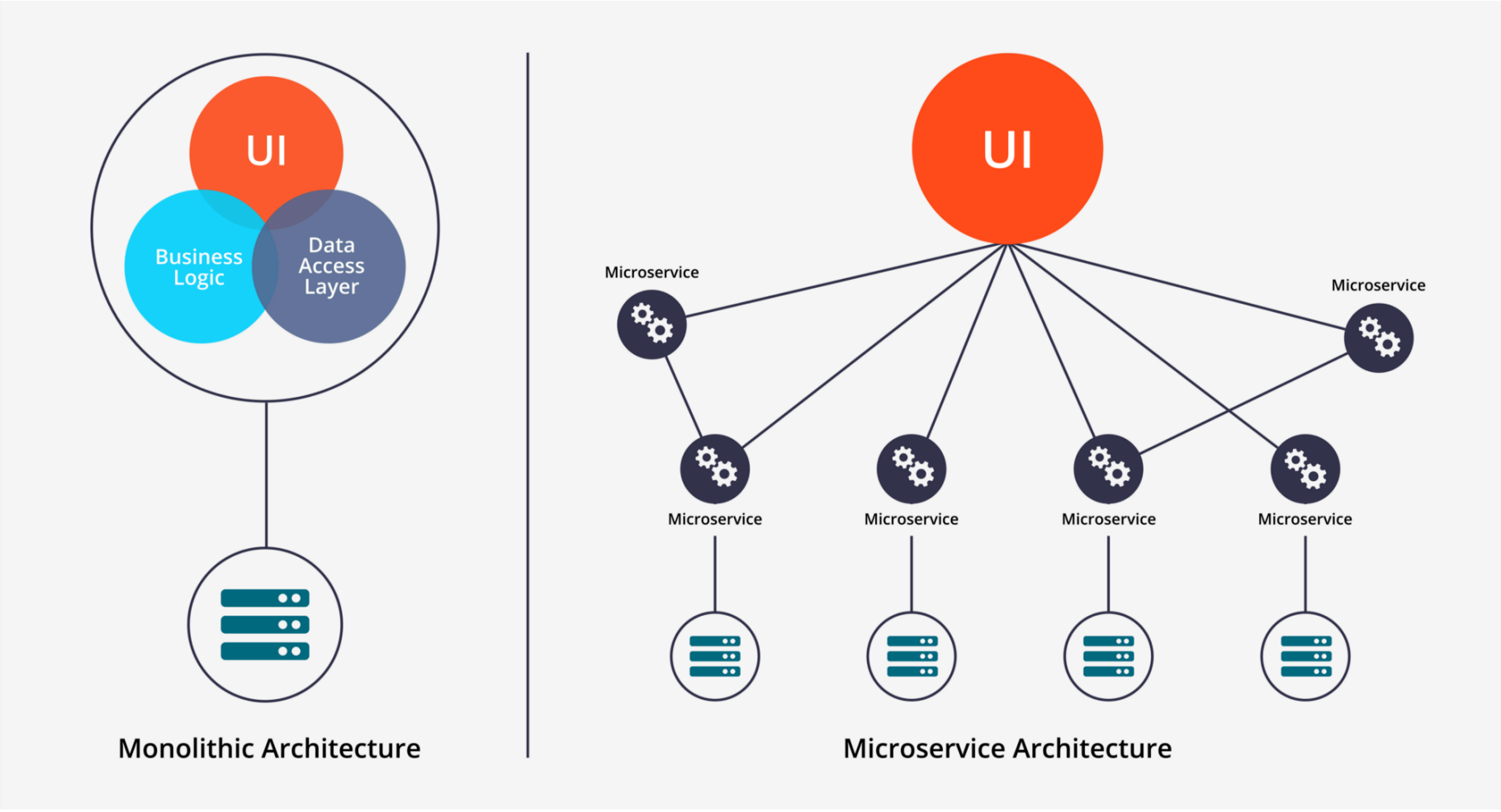
- Microservicios
- Frameworks JS
- Intro Vue

Arquitectura de Microservicios

Microservicios

- Las aplicaciones se **dividen en sus componentes más pequeños**, independientes entre sí.
- A diferencia del enfoque tradicional y monolítico de las aplicaciones, en el que todo se encuentra en una **única pieza**.
- Los microservicios funcionan en conjunto para llevar a cabo las **mismas tareas** que la aplicación monolítica.
- Los microservicios **facilitan la escalabilidad** de todo el sistema, se despliegan según se vayan necesitando.
- Pueden tener **distintas tecnologías** entre sí.
- Al ser más pequeños, son **más simples de mantener** y actualizar.

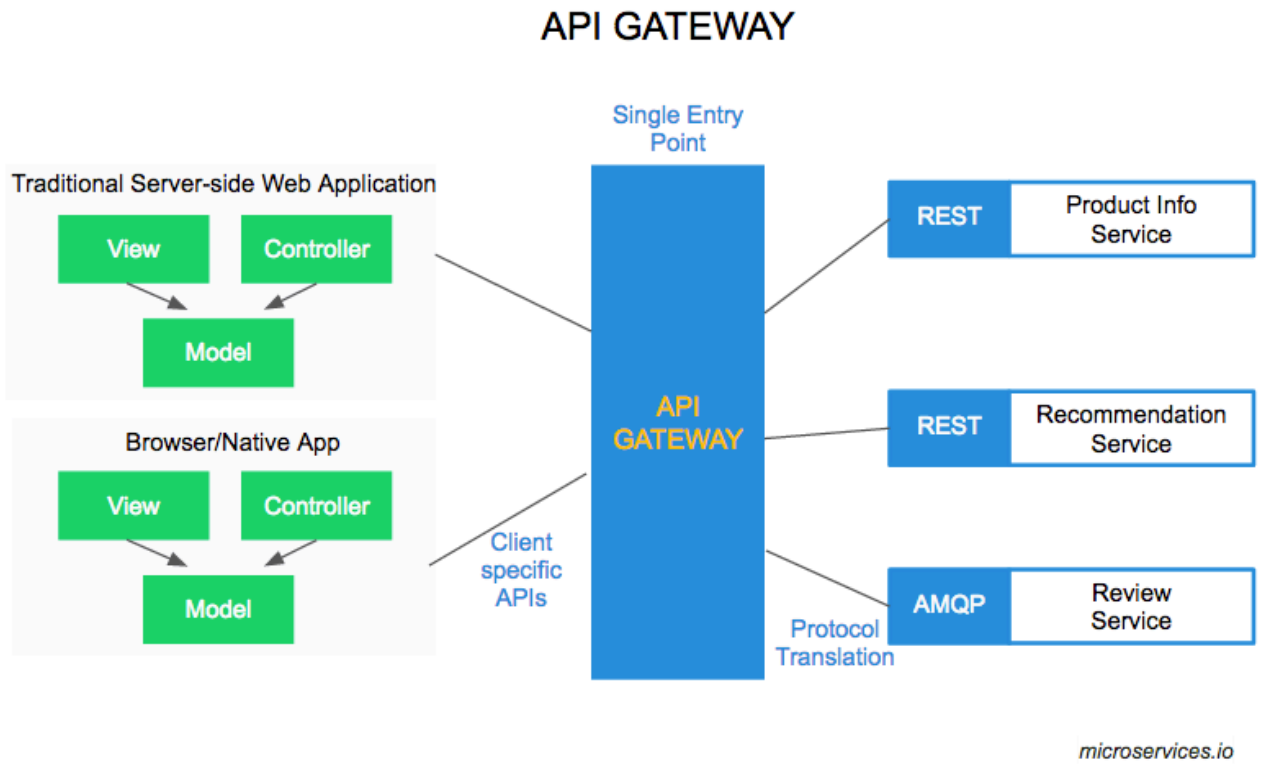
Arquitectura Monolítica vs Arquitectura de Microservicios



¿Cómo comunico un servicio con otro?

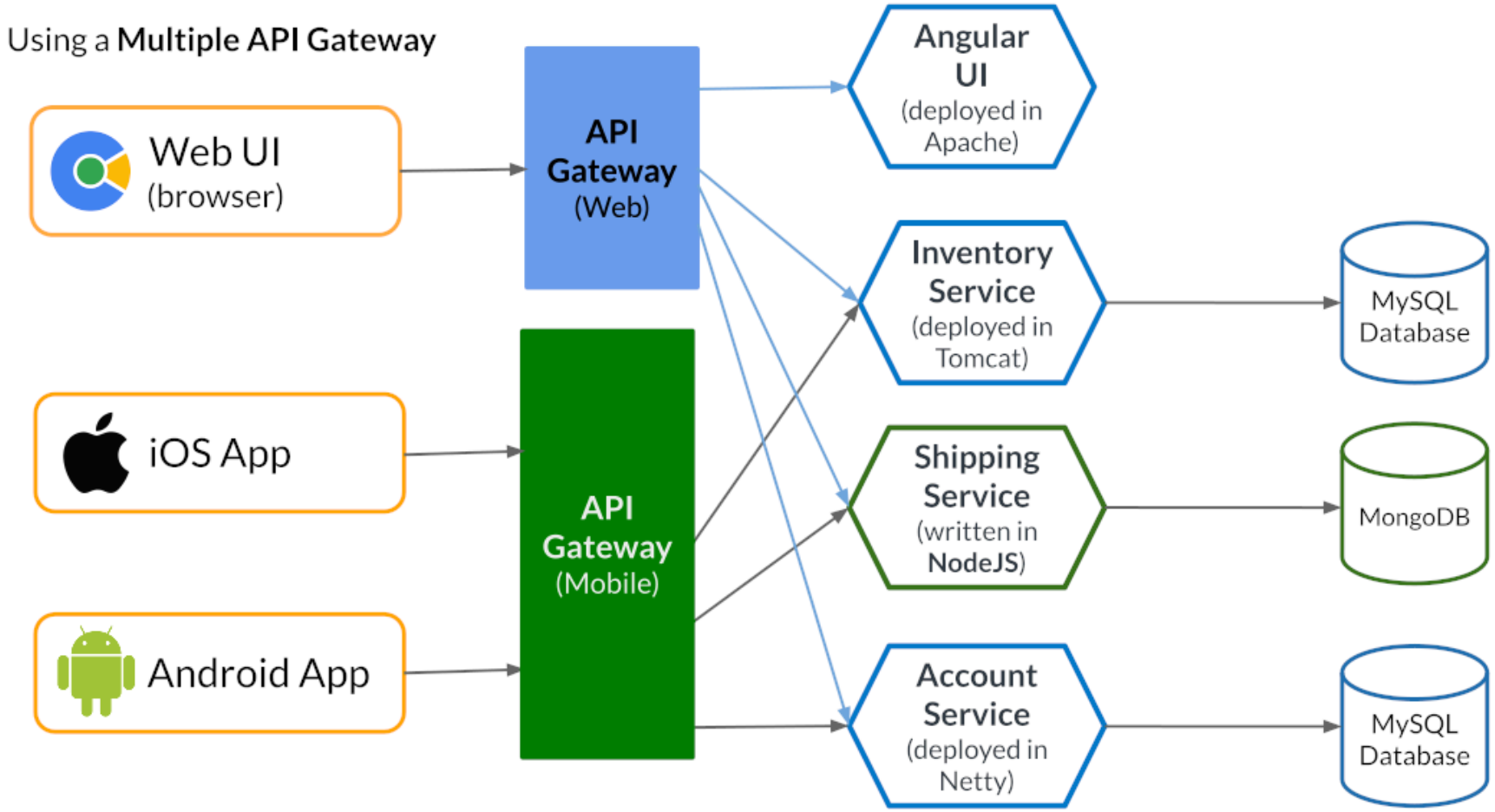
- Actualmente la opción más utilizada es mediante **APIs HTTP/REST** con JSON.
- Incluso puede centralizarse la comunicación utilizando un **API Gateway**.
- En dicho API Gateway puede implementarse una **capa de seguridad**, que ante una petición verifique si el cliente tiene permisos de acceso.

Microservicios: API Gateway

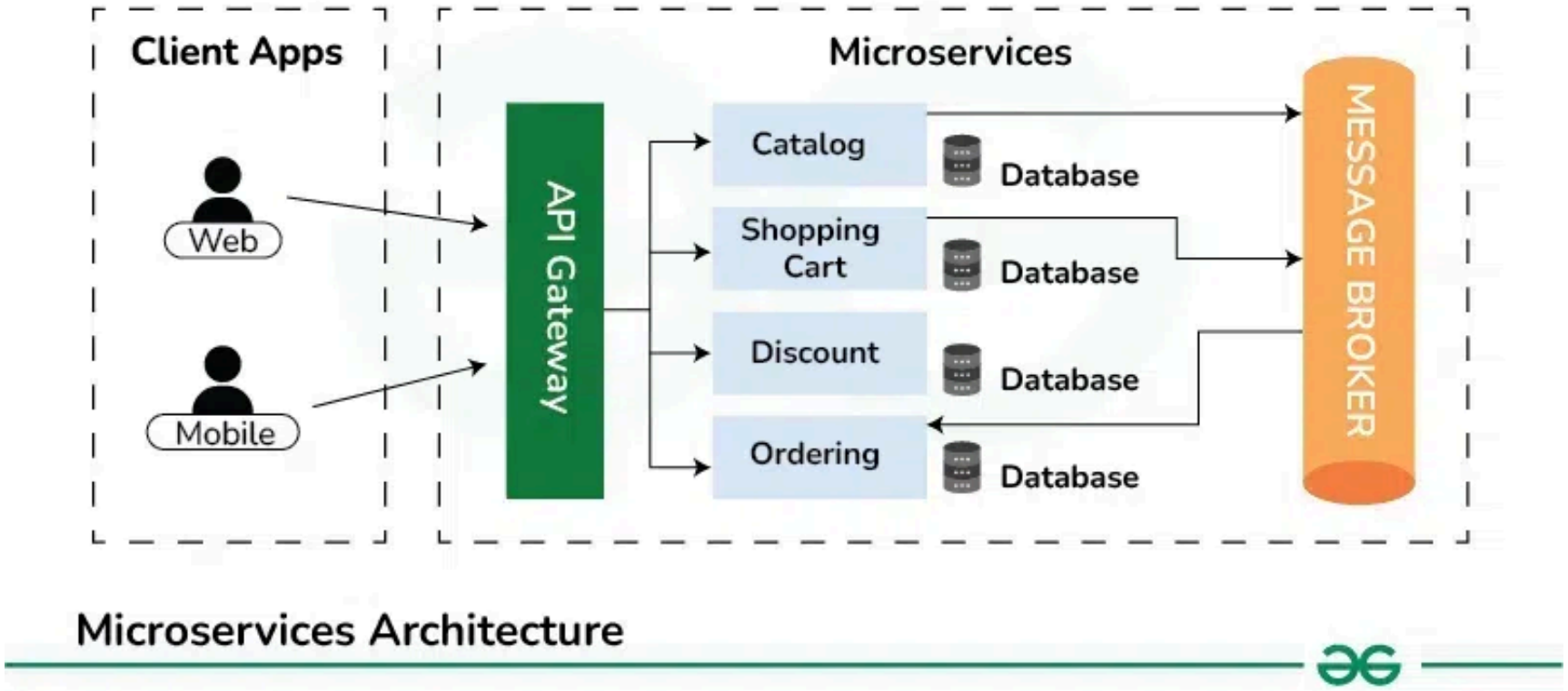


Patrón API Gateway (microservices.io)

Microservicios: Múltiples API Gateway



Microservicios: Message Broker



Patrón Messaging (microservices.io)

Microservicios: no todo son ventajas

- **La red falla**: lo que antes era una llamada a una función ahora es un request HTTP, que puede tardar o no llegar.
- **Datos repartidos**: cada servicio con su base de datos. Ya no hay una transacción que abarque todo.
- **Depurar es más difícil**: un error puede atravesar varios servicios. Hacen falta logs centralizados y trazas.
- **Más para operar**: más contenedores, pipelines, versiones de APIs y monitoreo.
- Suelen tener sentido con **muchos equipos** trabajando en paralelo. Ver Microservice trade-offs y Monolith first.

Pensemos: ¿y el sistema del Hospital?

Pedidos de comida: **solicitud** → **autorización** → **entrega**, con reportes y un kiosco en tablets.

¿Qué arquitectura elegirían? **Votemos.**

- **A.** Una aplicación Flask que arma todas las páginas con templates.
- **B.** Un backend Flask con una API JSON y un frontend Vue separado.
- **C.** Microservicios: pedidos, usuarios, reportes y kiosco, cada uno por su lado.

Lo que pide el enunciado: opción B

- **Un solo backend** Flask + PostgreSQL: un equipo chico, un dominio acotado. Microservicios agregaría costos sin beneficio.
- Pero el **frontend se separa**: Vue consume la API JSON.
- La misma API puede servir a **distintos clientes**: el sistema de gestión y el kiosco de las tablets.
- Separar front y back **ya es una decisión de arquitectura**: por eso necesitamos un framework JS.

Para seguir leyendo de Microservicios

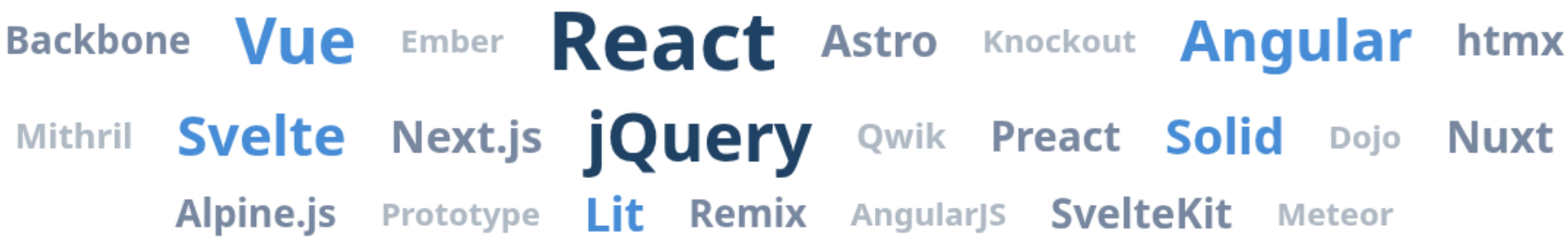
- Patrones de microservicios (microservices.io), de Chris Richardson.
- Guía de microservicios (Martin Fowler).
- Monolito vs. microservicios (Atlassian).
- AWS: https://aws.amazon.com/es/microservices/
- Redhat: https://www.redhat.com/es/topics/microservices
- Video en español: https://www.youtube.com/watch?v=9R2hFwIPGnQ

Frameworks JS

¿Librería o framework?

- Trabajar con **JS pelado** es complejo. Por eso nació **jQuery** (2006): una **librería** que simplifica el DOM y AJAX.
- **Librería**: nuestro código **la llama** cuando la necesita (jQuery, Axios).
- **Framework**: define la estructura de la aplicación y **llama a nuestro código** (Angular, Vue).
- jQuery sigue estando **muy** extendido, pero las aplicaciones web implementan **cada vez más funcionalidades y complejidad**, y una librería de atajos se queda corta.

Demasiadas librerías y frameworks JS

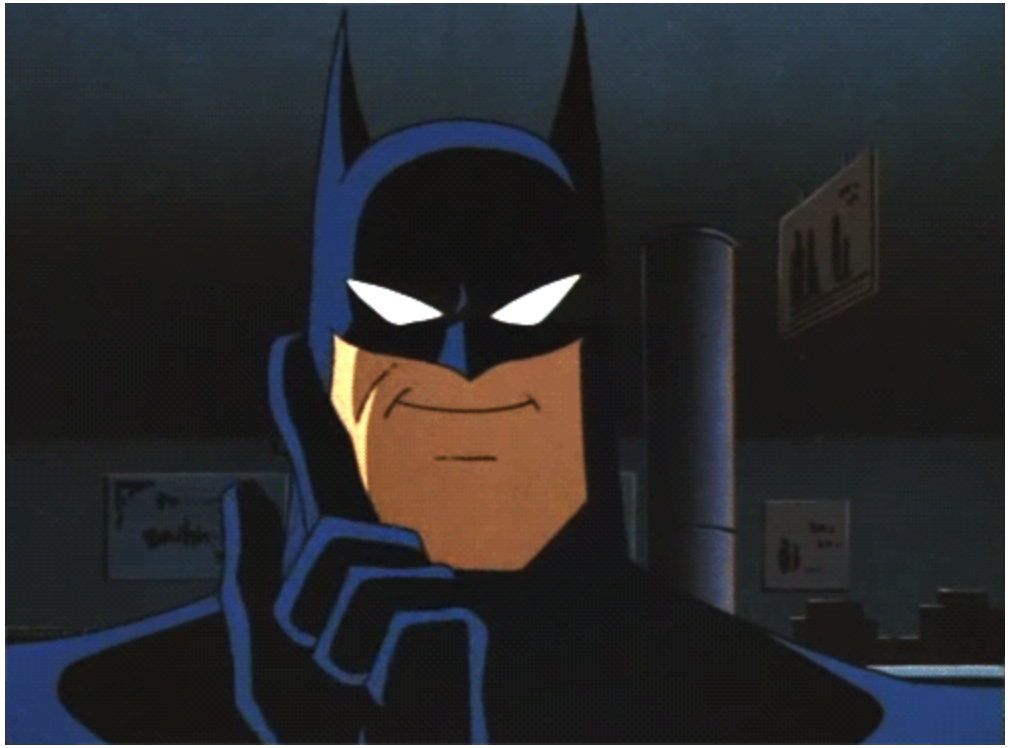


- **Librerías**: jQuery, Preact, Lit…
- **Frameworks**: Angular, Vue, Svelte, Solid…
- **Meta-frameworks** (construidos sobre un framework): Next.js, Nuxt, SvelteKit…
- Lista (incompleta) en Wikipedia.

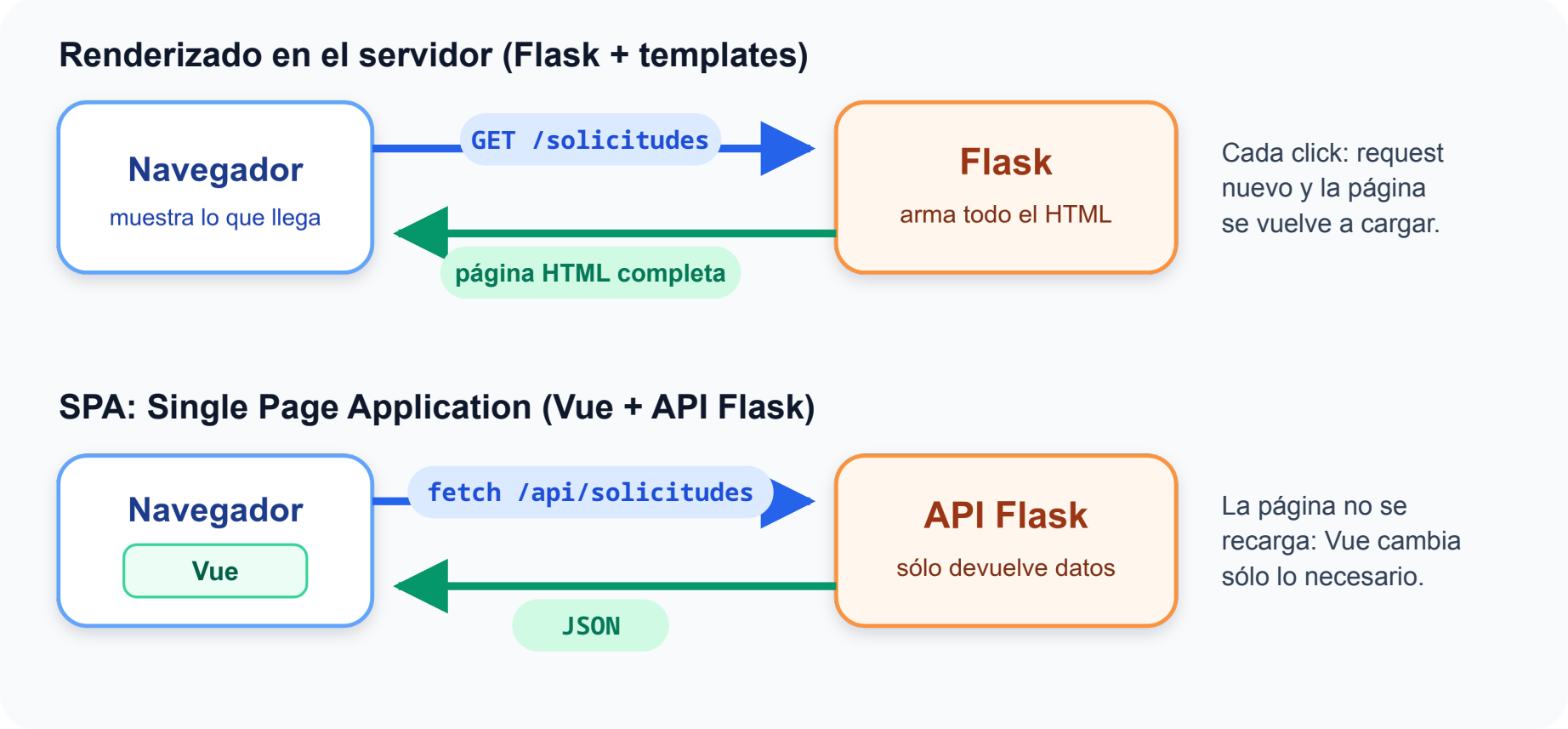
¿Por qué existen los frameworks JS?

Será porque ...

- Están basados en componentes.
- Tienen una comunidad sólida.
- Tienen muchas bibliotecas y componentes de terceros para manejar diferentes tareas.
- Tienen extensiones de navegador que ayudan a depurar.
- Son buenos para crear aplicaciones SPA (single page applications).
- Poseen una buena interfaz de línea de comandos (CLI) para facilitar el desarrollo.
- Realizan optimizaciones para mejorar el rendimiento.



Primero: cambió dónde vive la aplicación



- En una SPA la página **no se recarga**: el estado vive en el navegador y la vista tiene que seguirlo.

La verdadera razón:



- Cada framework lo resuelve a su manera: **Virtual DOM**, **signals**, compilación...
  Los signals incluso tienen una propuesta para el estándar de JS.
- Artículo Medium

Veámoslo: solicitudes sin framework

- Veamos sin_framework. Agreguemos 3 solicitudes y entreguemos una. **¿Dónde está el error?**

```
document.getElementById('agregar').addEventListener('click', () => {
  const texto = input.value
  solicitudes.push(texto)
  const li = document.createElement('li')
  // ... botón "Entregar"
  boton.addEventListener('click', () => {
    solicitudes.splice(solicitudes.indexOf(texto), 1)
    li.remove()
  })
  lista.appendChild(li)
  contador.textContent = solicitudes.length
})
```

- Y si en la consola ejecutamos `solicitudes.push('Dieta blanda')`, ¿qué pasa en la pantalla?

Sin framework: estado y DOM por separado

- Los datos (`solicitudes`) y la pantalla (`<li>`, `contador`) son **dos copias** de la misma información.
- **Cada** cambio de datos tiene que actualizar **a mano** cada parte del DOM que depende de ellos.
- Una actualización olvidada no da error: la pantalla **muestra algo falso**.
- Con más pantallas, más datos y más personas tocando el código, se vuelve inmanejable.

Angular, React y Vue: ¿cuál elegir?

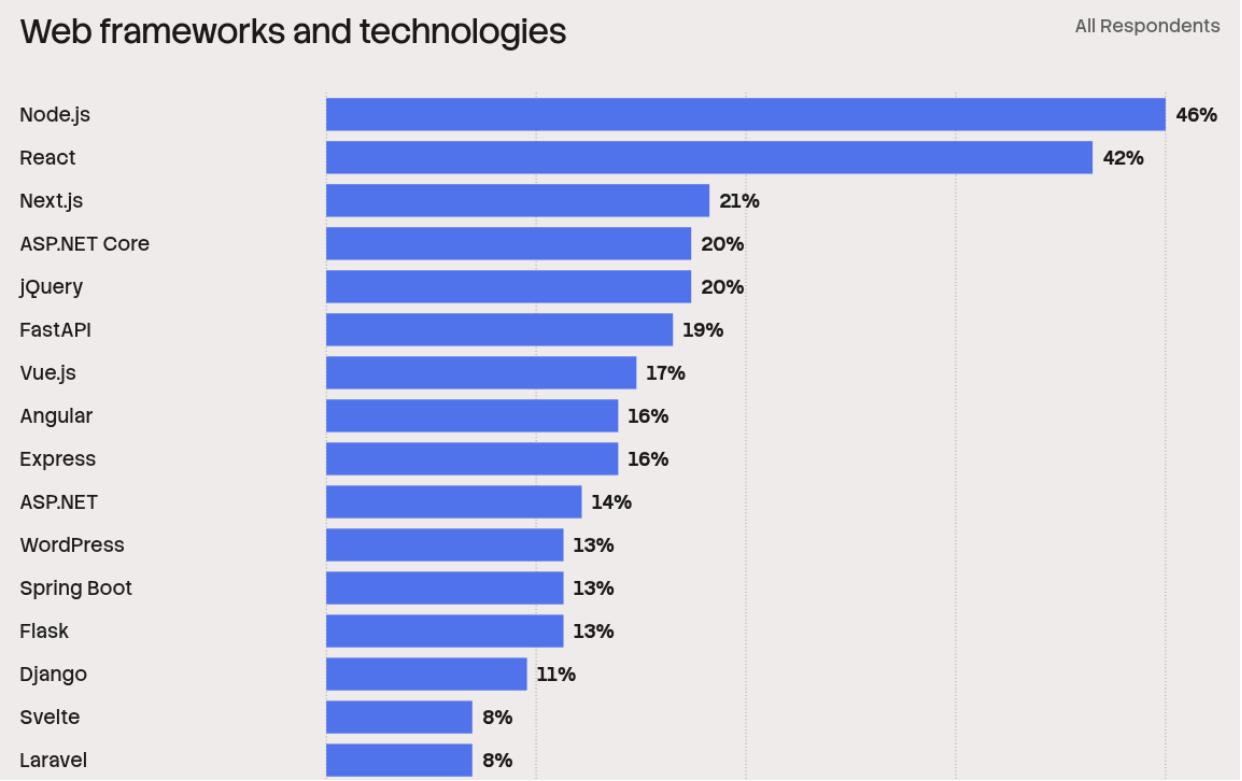
- Los tres resuelven **el mismo problema**: mantener la vista sincronizada con el estado.
- Se diferencian en:
  - **Cuánto incluyen**: framework completo o sólo la capa de vista.
  - **Cómo se escriben las vistas**: templates HTML o JSX.
  - **Cómo detectan los cambios** y actualizan el DOM.
- Para elegir importan el **equipo**, la **curva de aprendizaje**, el **ecosistema** y el **uso en la industria**.
- En el proyecto usamos **Vue 3**.

Angular vs. React vs. Vue

	Angular	React	Vue
Tipo	Framework completo	Librería de UI	Framework progresivo
Mantenido por	Google	Meta	Evan You + equipo
Vistas	Templates HTML + TypeScript	JSX	Templates HTML + JS/TS
Actualización del DOM	Detección de cambios / signals	Virtual DOM	Reactividad + Virtual DOM
Ruteo y estado	Incluidos	De terceros	Oficiales (Vue Router, Pinia)
Curva de aprendizaje	Alta	Media	Suave
Uso (SO 2026)	16,0 %	41,5 %	17,2 %

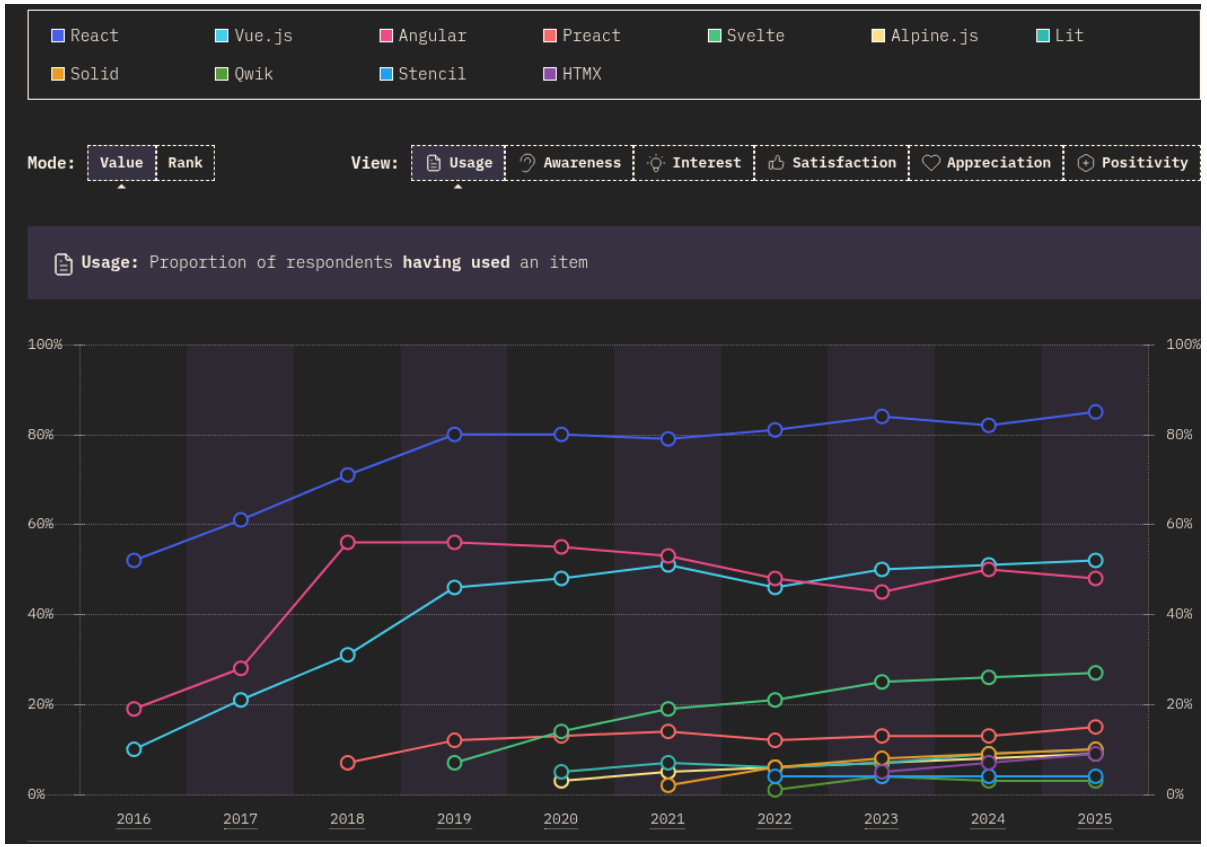
¿Qué se usa? Stack Overflow 2026

- Frameworks **y tecnologías** web más usados: mezcla front-end (React, Vue), back-end (FastAPI, Express, Flask, Django) y librerías (jQuery). Ver encuesta.

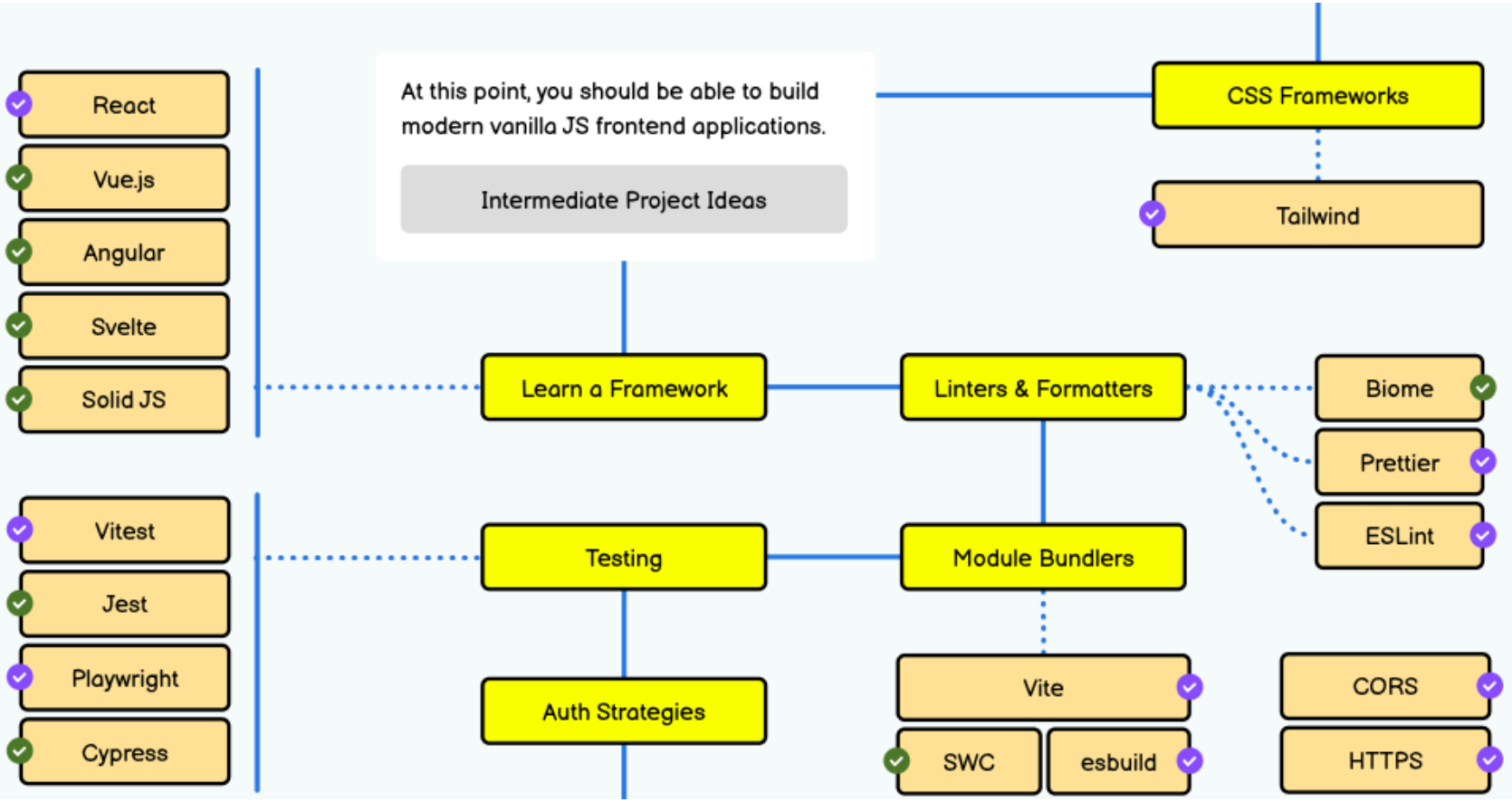


¿Qué se usa? State of JS 2025

- Proporción de personas que usaron cada framework front-end, según State of JavaScript 2025.



¿Dónde encaja un framework? Front-end roadmap



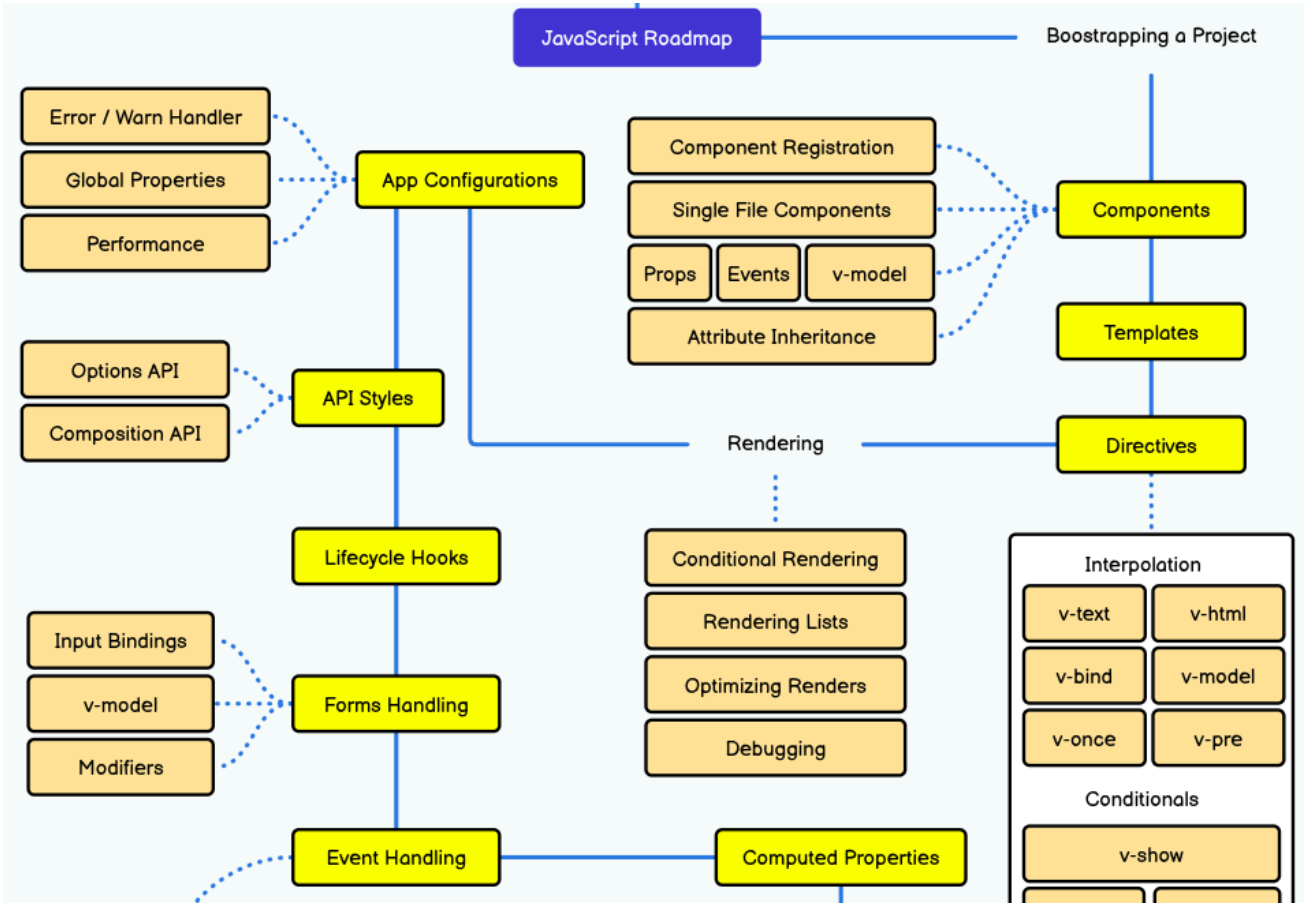
- Después de HTML, CSS, JS y Git. Ver roadmap completo en roadmap.sh/frontend.

Para seguir leyendo: JS Frameworks

- Entendiendo los frameworks JS (MDN)
- Reactividad en JS
- Reactividad en profundidad (Vue)
- Benchmark de frameworks JS

Vue.js

Vue.js roadmap: lo que vamos a ver



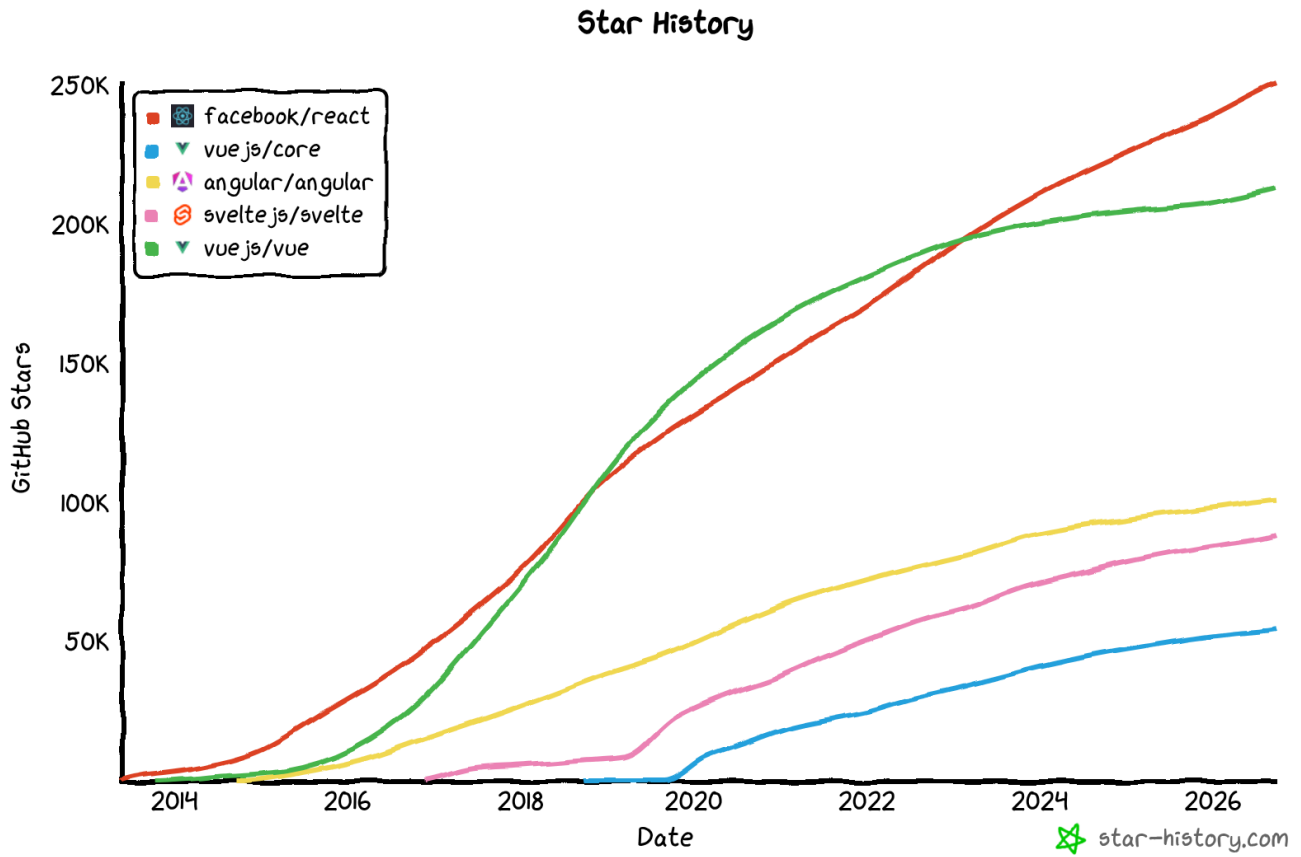
- Roadmap completo en roadmap.sh/vue.

Características

- Vue.js es un framework de JavaScript para front-end.
- **Curva de aprendizaje suave**: usa templates HTML con JavaScript.
- **Buen rendimiento**: un compilador optimiza los templates. Ver benchmark.
- Vue.js es un framework **progresivo**. Tiene la facilidad para usarlo y adaptarlo a proyectos tanto grandes como pequeños.
- Ha tenido un gran crecimiento y tiene una comunidad muy activa.

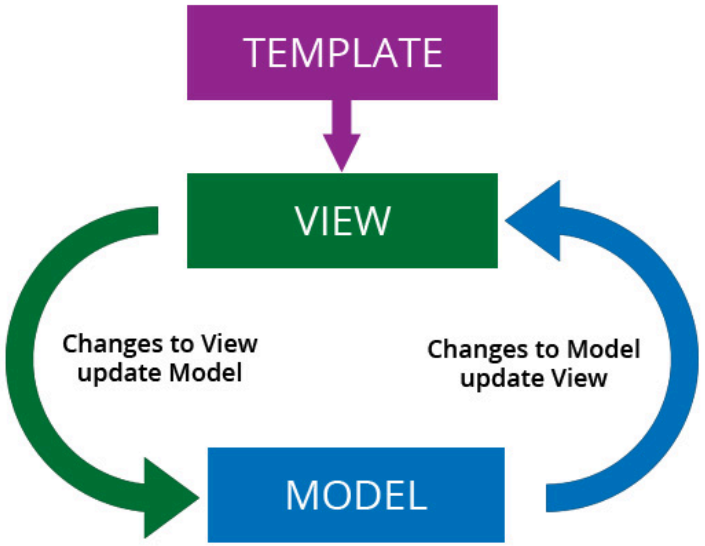
Comunidad de Vue en GitHub

- Estrellas según star-history.com. Ver la colección de frameworks.
- Vue 2 está en **vuejs/vue** y Vue 3 en **vuejs/core**: sus estrellas están repartidas.



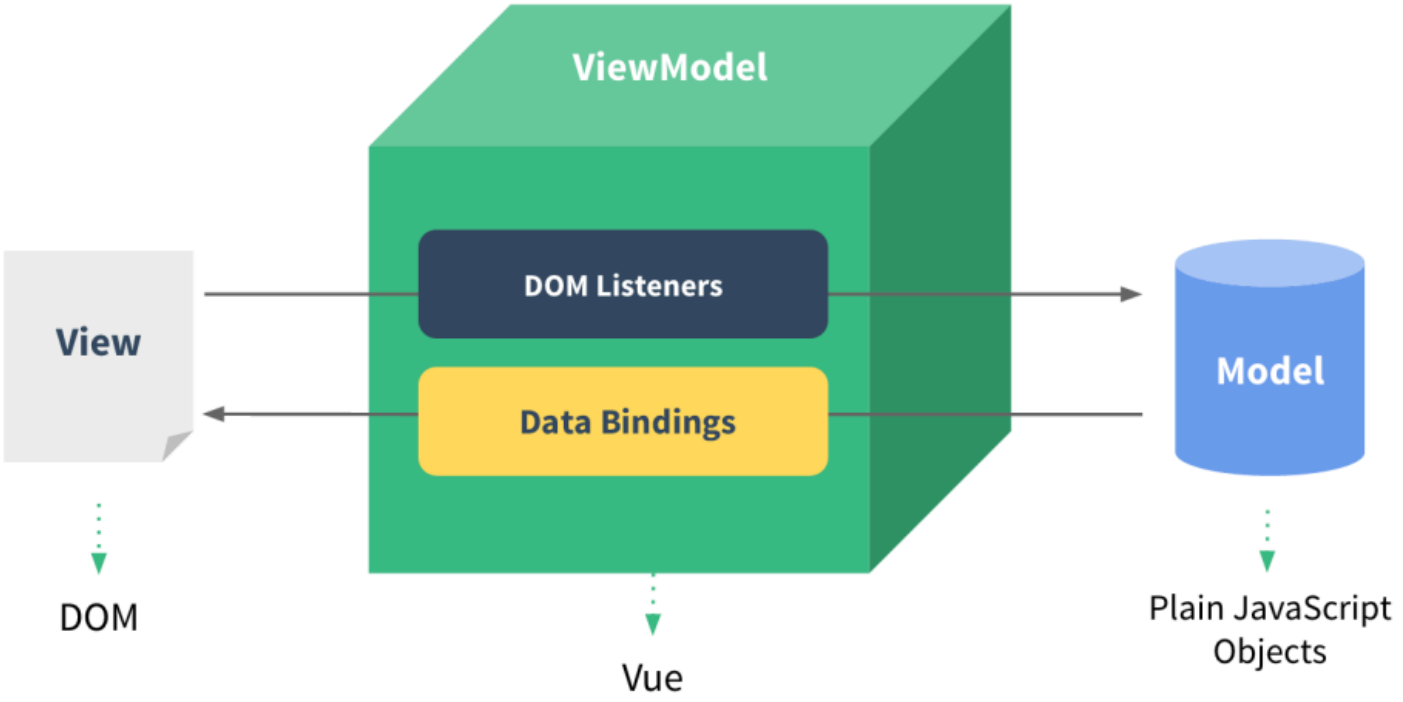
Reactividad y two-way data-binding

- Es un framework **reactivo**: cuando cambian los datos, Vue actualiza la vista automáticamente.
- En formularios, `v-model` agrega el camino inverso: lo que se escribe en un input actualiza los datos (**two-way data-binding**).



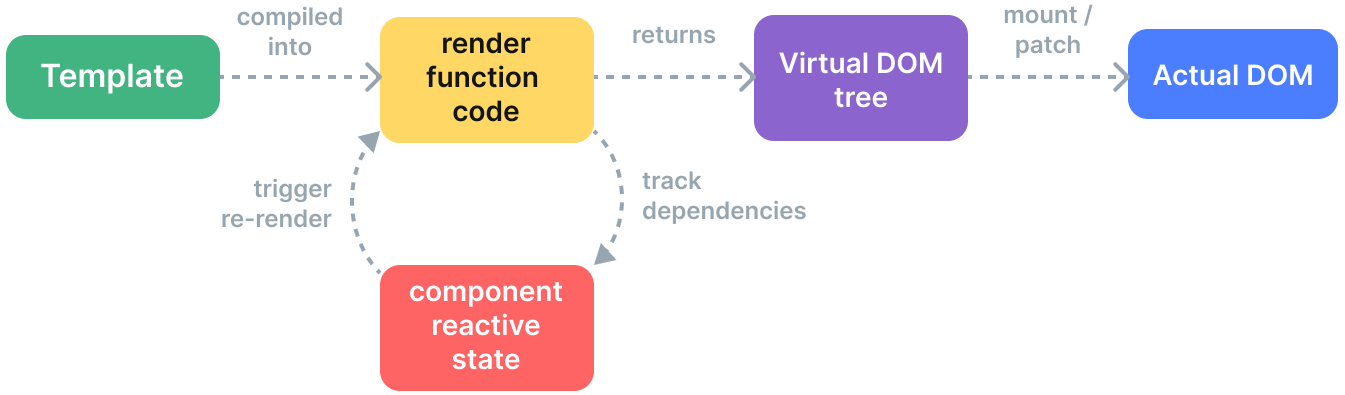
Patrón Model-View-ViewModel (MVVM)

- **Model**: los datos, objetos JS planos. **View**: el DOM. **ViewModel**: la instancia de Vue, que los conecta con *data bindings* y *DOM listeners*.



¿Cómo llega un cambio al DOM? Virtual DOM

- Modificar el DOM real es costoso. Vue (como React) trabaja sobre un **Virtual DOM**: una representación liviana del DOM hecha con objetos JS.
- El template se compila a una **función de render**, que devuelve el Virtual DOM. Cuando cambia el estado reactivo, se vuelve a ejecutar y Vue aplica al DOM real sólo las diferencias (*patch*).



- El compilador marca las partes estáticas del template para que la comparación las saltee. Ver mecanismo de renderizado.

Integración y herramientas

- Vue.js está más enfocado hacia la vista, y **puede ser implementado en el HTML de cualquier proyecto web** sin requerir cambios drásticos.
- Existe la extensión **Vue DevTools** para navegadores (también como plugin de Vite): devtools.vuejs.org. Nos asiste en el desarrollo.
- Vue 3 soporta los navegadores modernos con soporte nativo de **ES2016** (no soporta IE11). Ver FAQ oficial y versiones de ECMAScript (MDN).

Instalación VueJS:

- Descargando el js e incluyéndolo directamente en un tag `<script>`.
- Linkear directamente desde una **CDN** (Content Delivery Network).

Vue 3 (Versión actual):

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

Vue 2 (Versión LTS - **finalizó el 31/12/2023**) - NO UTILIZAR:

```
<script src="https://cdn.jsdelivr.net/npm/vue@2"></script>
```

Instalación VueJS:

- Instalar Vue via **npm** (manejador de paquetes por defecto para Node.js):

```
npm create vue@latest
```



Se utiliza un setup guiado basado en Vite una herramienta para construir código web moderno.

```
cd NOMBRE_PROYECTO
npm install
npm run dev
```

- Es la forma que vamos a usar en el **proyecto**.
- Node.js versión requerida **^22.18.0 || >=24.12.0**.

Incluyéndolo de una CDN:

Veamos hello_world. Los datos y DOM están ahora relacionados utilizando **{{}}**. Modifiquemos **vue_app.message**.

```
  <title>Mi primera aplicaci&oacute;n Vue</title>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
  <div id="app"> <h2>{{ message }}</h2> </div>
  <script>
    const HelloVueApp = {
      data() {
        return {
          message: 'Hola Vue!!!'
        }
      }
    }
   vue_app = Vue.createApp(HelloVueApp).mount('#app')
  </script>
```

Antes de probar: ¿qué pasa?

Con la página abierta, en la consola ejecutamos:

```
vue_app.message = 'Chau jQuery'
```

- **A.** Nada: hay que recargar la página.
- **B.** El `<h2>` muestra el texto nuevo.
- **C.** Error: `message` no se puede modificar desde afuera.

Ejemplo Two way data-binding:

- Hace la relación bidireccional entre un input y los datos de la aplicación Vue.
- Veamos v-model.

```
<div id="two-way-binding">
  <p>{{ message }}</p>
  <input v-model="message">
</div>
<script>
  const App = {
    data() {
      return {
        message: 'Bienvenidos a Proyecto!!'
      }
    }
  }
 vue_app = Vue.createApp(App).mount('#two-way-binding')
</script>
```

Volvamos a las solicitudes, ahora con Vue

- Veamos con_vue: agreguemos y entreguemos. Y en la consola: `vue_app.solicitudes.push({ id: 1, texto: 'Dieta blanda' })`.

```
<h2>Pendientes: {{ solicitudes.length }}</h2>
<ul>
  <li v-for="s in solicitudes" :key="s.id">
    {{ s.texto }} <button @click="entregar(s)">Entregar</button>
  </li>
</ul>
```

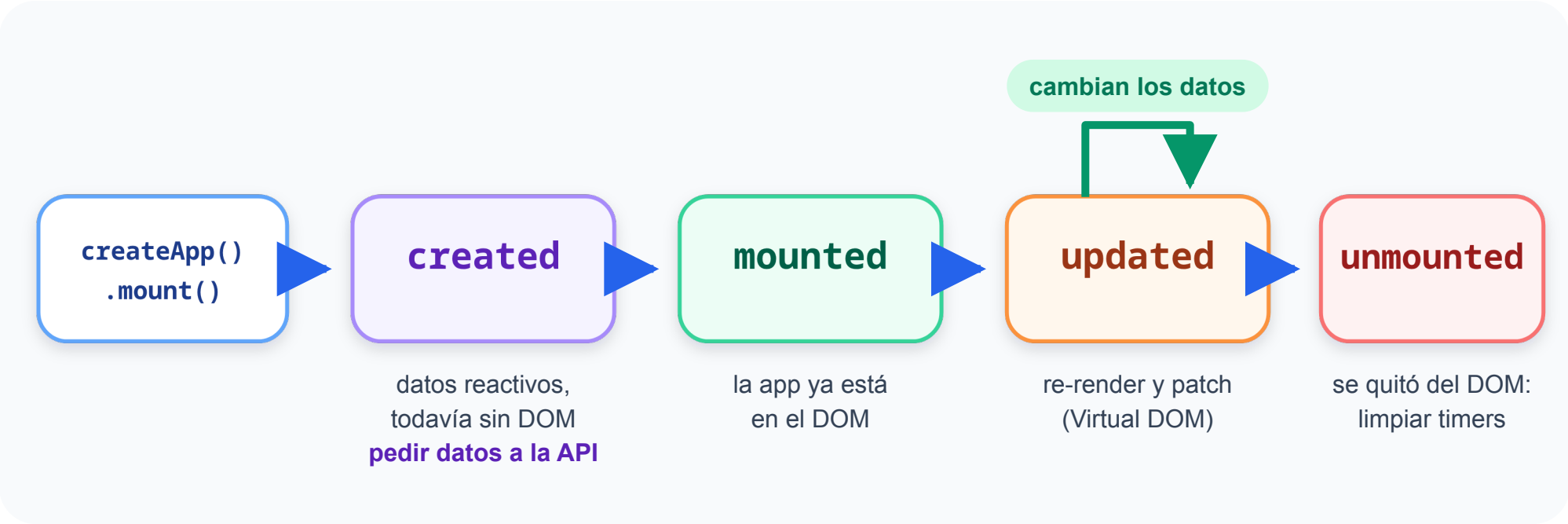
```
entregar(s) {
  this.solicitudes.splice(this.solicitudes.indexOf(s), 1)
}
```

- Sólo cambiamos los **datos**; el contador y la lista se actualizan solos. La sintaxis (`v-for`, `@click`) la vemos en el video de directivas.

Mirando el estado con Vue DevTools

- Con la extensión Vue DevTools abrimos la pestaña **Vue** en las herramientas del navegador, sobre v-model o con_vue:
  - El árbol de la aplicación y su **estado** (`message`, `solicitudes`).
  - Editamos un dato **desde el panel** y la vista cambia.
  - Escribimos en el input y el dato cambia en el panel.
- Es la herramienta para responder **"¿qué datos tiene mi aplicación ahora?"** en vez de llenar el código de `console.log`.
- ¿No aparece la pestaña? Si abrimos el HTML como archivo (`file://`), hay que habilitar **"Permitir el acceso a URLs de archivo"** en los detalles de la extensión, o servir la carpeta con `python3 -m http.server`.

Adelanto: el ciclo de vida de una app Vue



- Vue ejecuta **hooks** en cada etapa. Veamos ciclo_de_vida con la consola abierta y, al final, `app.unmount()`.
- En el próximo encuentro lo usamos para cargar datos de la API al mostrar un componente.

¿Dudas?

Para la próxima: video de directivas

- **Directivas**: `v-bind`, `v-if`, `v-for`, `v-on`, `v-model`.
- **Propiedades computadas** y **watchers**.
- **Consumir una API** con `fetch` y `axios`.
- **Ciclo de vida** de una instancia Vue.
- En el próximo encuentro: **componentes**.

Referencias Vue

- Guía oficial Vue: https://vuejs.org/guide/introduction.html